



Module Objective: In previous weeks, we focused on **Exploration** (finding a path using algorithms like A*). Today, we shift to **Validation**. We will use Logical Reasoning to enforce constraints, ensuring that the actions our agent (Drow Ranger) wants to take are actually safe and legally permitted by the game rules.

Part 1: Reachability vs. Feasibility

1.1 Evaluating State Constraints (Analyze)

Your agent is located at the Mid_Tower. The A* Search algorithm has found a physical path to the River_Rune. However, the agent's Knowledge Base (KB) contains the following strict logical constraint:

"If the map is completely dark (no wards) and the enemy Bloodseeker is missing, the River is Unsafe."

Current sensors indicate: DarkMap = True and BloodseekerMissing = True.

Question: Is the River_Rune state *Reachable*? Is it *Feasible*? Explain the difference based on the lecture.

- THE CONSTRAINT WRITTEN FORMALLY IS: DARKMAP AND BLOODSEEKERMISSING => UNSAFE(RIVER_RUNE).
- THE SENSORS REPORT DARKMAP = TRUE AND BLOODSEEKERMISSING = TRUE, SO BOTH PREMISES OF THE RULE HOLD. BY MODUS PONENS THE KNOWLEDGE BASE THEREFORE ENTAILS UNSAFE(RIVER_RUNE) = TRUE.
- IS IT REACHABLE? YES. A* HAS FOUND A PHYSICALLY CONNECTED, TRAVERSABLE PATH FROM MID_TOWER TO RIVER_RUNE, SO THE STATE EXISTS IN THE SEARCH SPACE AND HAS A FINITE-COST PATH LEADING TO IT.
- IS IT FEASIBLE? NO. THE KNOWLEDGE BASE ENTAILS THAT THE RIVER IS UNSAFE, SO THE STATE VIOLATES A HARD LOGICAL CONSTRAINT AND THE ACTION IS NOT LEGALLY PERMITTED, EVEN THOUGH IT IS PHYSICALLY POSSIBLE.
- REACHABILITY IS A PROPERTY OF THE SEARCH SPACE. IT IS DECIDED BY THE SEARCH ALGORITHM SUCH AS A*, BFS OR UCS USING THE TRANSITION MODEL, AND IT ONLY ANSWERS THE QUESTION "DOES SOME SEQUENCE OF ACTIONS CONNECT MY CURRENT STATE TO THIS STATE?"
- FEASIBILITY IS A PROPERTY OF THE CONSTRAINTS. IT IS DECIDED BY THE INFERENCE ENGINE REASONING OVER THE KNOWLEDGE BASE, AND IT ANSWERS THE QUESTION "DOES THIS STATE VIOLATE ANYTHING I ALREADY KNOW TO BE TRUE?"

- THE KEY DIFFERENCE IS THAT A STATE CAN BE REACHABLE AND STILL BE INFEASIBLE, WHICH IS EXACTLY THE SITUATION HERE. A PURELY SEARCH-DRIVEN AGENT WOULD WALK STRAIGHT INTO A DEATH TRAP, WHICH IS WHY A KNOWLEDGE-BASED REASONING LAYER MUST SIT ON TOP OF SEARCH. SEARCH PROPOSES A PLAN, LOGIC VALIDATES OR REJECTS IT.
- TO MAKE THE RIVER FEASIBLE THE AGENT MUST FALSIFY ONE OF THE PREMISES: PLACE A WARD SO THAT DARKMAP BECOMES FALSE, OR WAIT UNTIL BLOODSEEKER REAPPEARS SO THAT BLOODSEEKERMISSING BECOMES FALSE. ONCE EITHER PREMISE FAILS THE RULE NO LONGER FIRES AND UNSAFE IS NO LONGER ENTAILED.

Part 2: Propositional Logic & CNF Conversion

2.1 Translation and CNF (Apply)

Let's define the following propositions:

- **S:** Drow has ShadowBlade.
- **M:** Drow has full Mana.
- **G:** Drow can safely Gank.

Step A: Translate this sentence into Propositional Logic: *"If Drow has a ShadowBlade and full Mana, then Drow can safely Gank."*

- THE TWO CONDITIONS ARE JOINED BY "AND", SO THEY FORM THE CONJUNCTION S AND M.
- THE WORD "THEN" MAKES THIS A CONDITIONAL, SO THE CONJUNCTION IMPLIES G.
- THE TRANSLATION IS: $(S \text{ AND } M) \Rightarrow G$

Step B: Convert your answer from Step A into **Conjunctive Normal Form (CNF)**. Show the mathematical steps (Implication Elimination and De Morgan's Law).

- STEP 1, IMPLICATION ELIMINATION. APPLY THE EQUIVALENCE $A \Rightarrow B$ IS THE SAME AS $(\text{NOT } A) \text{ OR } B$. THIS GIVES $\text{NOT}(S \text{ AND } M) \text{ OR } G$.
- STEP 2, DE MORGAN'S LAW. APPLY THE EQUIVALENCE $\text{NOT}(A \text{ AND } B)$ IS THE SAME AS $(\text{NOT } A) \text{ OR } (\text{NOT } B)$ TO PUSH THE NEGATION INWARDS. THIS GIVES $(\text{NOT } S \text{ OR } \text{NOT } M) \text{ OR } G$.
- STEP 3, ASSOCIATIVITY OF OR. THE BRACKETS ARE REDUNDANT ACROSS A CHAIN OF DISJUNCTIONS, SO THEY ARE DROPPED. THIS GIVES $\text{NOT } S \text{ OR } \text{NOT } M \text{ OR } G$.
- FINAL CNF ANSWER: $\text{NOT } S \text{ OR } \text{NOT } M \text{ OR } G$
- THIS IS A CONJUNCTION CONTAINING A SINGLE DISJUNCTIVE CLAUSE, SO NO DISTRIBUTION STEP WAS REQUIRED. IT IS ALSO A HORN CLAUSE, BECAUSE IT CONTAINS EXACTLY ONE POSITIVE LITERAL, G, WHICH IS WHAT ALLOWS FORWARD AND BACKWARD CHAINING TO BE USED ON IT IN PART 5.

Part 3: Proof by Resolution

3.1 Mathematical Contradiction (Evaluate)

The Inference Engine needs to definitively prove if Drow can safely Gank (**G**). Your current Knowledge Base (in CNF) is:

1. $\sim S \vee \sim M \vee G$ (Rule from Part 2)
2. S (Sensor fact: ShadowBlade is in inventory)
3. M (Sensor fact: Mana is at 100%)

Use **Proof by Contradiction** using the Resolution rule to prove **G**. Write out your resolution steps (e.g., what you add to the KB, and which clauses you smash together until you hit an empty clause).

- THE GOAL IS TO PROVE THAT THE KNOWLEDGE BASE ENTAILS **G**. RESOLUTION DOES THIS BY REFUTATION: WE ASSUME THE OPPOSITE, ADD IT TO THE KNOWLEDGE BASE, AND SHOW THE RESULT IS UNSATISFIABLE BY DERIVING THE EMPTY CLAUSE.
- STARTING CLAUSE 1: NOT S OR NOT M OR G, WHICH IS THE RULE DERIVED IN PART 2.
- STARTING CLAUSE 2: S, THE SENSOR FACT THAT SHADOWBLADE IS IN THE INVENTORY.
- STARTING CLAUSE 3: M, THE SENSOR FACT THAT MANA IS AT 100%.
- WHAT IS ADDED TO THE KNOWLEDGE BASE IS CLAUSE 4: NOT G. THIS IS THE NEGATION OF THE QUERY, MEANING WE ASSUME FOR CONTRADICTION THAT DROW CANNOT SAFELY GANK.
- RESOLUTION STEP 1. RESOLVE CLAUSE 1 WITH CLAUSE 2 ON THE COMPLEMENTARY LITERALS NOT S AND S. THIS CANCELS S AND PRODUCES CLAUSE 5: NOT M OR G.
- RESOLUTION STEP 2. RESOLVE CLAUSE 5 WITH CLAUSE 3 ON THE COMPLEMENTARY LITERALS NOT M AND M. THIS CANCELS M AND PRODUCES CLAUSE 6: G.
- RESOLUTION STEP 3. RESOLVE CLAUSE 6 WITH CLAUSE 4 ON THE COMPLEMENTARY LITERALS G AND NOT G. THIS CANCELS G AND PRODUCES CLAUSE 7, WHICH IS THE EMPTY CLAUSE.
- DERIVING THE EMPTY CLAUSE MEANS THAT THE KNOWLEDGE BASE TOGETHER WITH NOT G IS A CONTRADICTION, SO THAT SET IS UNSATISFIABLE.
- SINCE ASSUMING NOT G LEADS TO A CONTRADICTION, NOT G MUST BE FALSE, AND THEREFORE G MUST BE TRUE.
- THE KNOWLEDGE BASE ENTAILS G. DROW CAN SAFELY GANK, AND THIS IS NOW PROVEN RATHER THAN ASSUMED, SO THE INFERENCE ENGINE CAN AUTHORISE THE GANK ACTION.

Part 4: First-Order Logic & Unification

4.1 Variable Binding (Understand)

Propositional logic forces us to write a separate rule for every single enemy. First-Order Logic (FOL) allows us to use variables. Consider this rule:

$$\forall x \text{ (Enemy}(x) \wedge \text{Invisible}(x) \Rightarrow \text{CarryDust}(\text{Agent}))$$

The agent's vision sensor detects an entity and generates the following raw percepts: $\text{Enemy}(\text{Spectre})$ and $\text{Invisible}(\text{Spectre})$.

What is the substitution (θ) required to **Unify** the sensor data with the general rule?

- **UNIFICATION MEANS FINDING A SUBSTITUTION THAT MAKES THE PREMISES OF THE RULE SYNTACTICALLY IDENTICAL TO THE FACTS COMING FROM THE SENSOR.**
- **UNIFYING ENEMY(X) WITH ENEMY(SPECTRE) REQUIRES BINDING THE VARIABLE X TO THE CONSTANT SPECTRE.**
- **UNIFYING INVISIBLE(X) WITH INVISIBLE(SPECTRE) REQUIRES THE SAME BINDING, X TO SPECTRE, SO THE TWO BINDINGS ARE CONSISTENT WITH EACH OTHER.**
- **THE REQUIRED SUBSTITUTION, WHICH IS THE MOST GENERAL UNIFIER, IS THEREFORE: $\theta = \{ X / SPECTRE \}$**
- **APPLYING θ TO THE RULE GIVES THE GROUND INSTANCE: ENEMY(SPECTRE) AND INVISIBLE(SPECTRE) $\Rightarrow$ CARRYDUST(AGENT).**
- **BOTH PREMISES OF THIS INSTANCE ARE NOW MATCHED BY FACTS ALREADY IN THE KNOWLEDGE BASE, SO GENERALIZED MODUS PONENS FIRES AND THE AGENT INFERS CARRYDUST(AGENT), MEANING IT SHOULD BUY A DUST OF APPEARANCE.**
- **THIS IS WHY FIRST-ORDER LOGIC MATTERS. IN PROPOSITIONAL LOGIC A SEPARATE RULE WOULD BE NEEDED FOR SPECTRE, RIKI, CLINKZ, WEAVER AND EVERY OTHER INVISIBLE HERO. WITH A SINGLE QUANTIFIED VARIABLE, ONE RULE COVERS EVERY CURRENT AND FUTURE INVISIBLE ENEMY, AND ONLY THE BINDING CHANGES.**

Part 5: Forward vs. Backward Chaining

5.1 Tracing Inference Directions (Analyze)

Consider an Expert System built with the following Horn Clauses:

1. $\text{HasDust} \wedge \text{TargetVisible} \Rightarrow \text{CanAttack}$
2. $\text{CanAttack} \Rightarrow \text{WinTeamFight}$

Trace 1: The agent's sensors suddenly register HasDust and TargetVisible. Explain how a **Data-Driven (Forward Chaining)** algorithm would process this.

- **ITERATION 0. WORKING MEMORY CONTAINS HASDUST AND TARGETVISIBLE, WHICH HAVE JUST BEEN ASSERTED INTO THE KNOWLEDGE BASE FROM THE PERCEPTS.**
- **ITERATION 1. THE ENGINE SCANS THE RULE BASE AND CHECKS THE PREMISES OF RULE 1. BOTH HASDUST AND TARGETVISIBLE ARE PRESENT IN WORKING MEMORY, SO RULE 1 IS SATISFIED AND FIRES. ITS CONCLUSION, CANATTACK, IS ADDED TO WORKING MEMORY AS A NEW FACT.**
- **ITERATION 2. THE ENGINE SCANS AGAIN WITH THE ENLARGED WORKING MEMORY. THE PREMISE OF RULE 2 IS CANATTACK, WHICH IS NOW A KNOWN FACT, SO RULE 2 FIRES AND ITS CONCLUSION WINTEAMFIGHT IS ADDED TO WORKING MEMORY.**
- **ITERATION 3. THE ENGINE SCANS ONCE MORE. NO RULE HAS PREMISES THAT ARE SATISFIED BUT WHOSE CONCLUSION IS STILL UNKNOWN, SO NO NEW FACTS CAN BE DERIVED AND THE**

ALGORITHM HALTS AT A FIXED POINT.

- THE OVERALL MECHANISM IS TO START FROM KNOWN DATA, REPEATEDLY MATCH RULE PREMISES, WHICH ARE THE LEFT-HAND SIDES, AGAINST WORKING MEMORY, FIRE EVERY MATCHING RULE, ADD THE CONCLUSIONS AS NEW FACTS, AND REPEAT UNTIL NOTHING NEW CAN BE PRODUCED.
- THE DIRECTION OF TRAVEL IS FROM FACTS UPWARDS TO CONCLUSIONS, WHICH IS WHY IT IS CALLED BOTTOM-UP OR DATA-DRIVEN REASONING.
- ITS CHARACTER IS OPPORTUNISTIC. NOBODY ASKED THE AGENT WHETHER IT COULD WIN THE TEAM FIGHT; THE INCOMING DATA VOLUNTEERED THAT CONCLUSION BY ITSELF.
- ITS MAIN COST IS THAT IT DERIVES EVERYTHING THAT IS DERIVABLE, INCLUDING CONCLUSIONS NOBODY ACTUALLY NEEDED, WHICH BECOMES WASTEFUL ON A LARGE KNOWLEDGE BASE.
- IT IS TYPICALLY USED FOR REACTIVE MONITORING, FOR EXAMPLE CONTINUOUSLY PROCESSING SENSOR UPDATES INSIDE A REAL-TIME GAME LOOP.

Trace 2: The overall objective given to the agent is to WinTeamFight. Explain how a **Goal-Driven (Backward Chaining)** algorithm would process this.

- THE ENGINE SETS WINTEAMFIGHT AS THE CURRENT GOAL AND FIRST CHECKS WHETHER IT IS ALREADY A KNOWN FACT. IT IS NOT.
- IT THEN SEARCHES THE RULE BASE FOR A RULE WHOSE CONCLUSION MATCHES THE GOAL. RULE 2 CONCLUDES WINTEAMFIGHT, SO THE ENGINE ADOPTS THAT RULE'S PREMISE, CANATTACK, AS A NEW SUB-GOAL.
- IT CHECKS WHETHER CANATTACK IS ALREADY A KNOWN FACT. IT IS NOT, SO IT SEARCHES AGAIN FOR A RULE CONCLUDING CANATTACK AND FINDS RULE 1.
- RULE 1 HAS TWO PREMISES, SO THESE BECOME TWO FURTHER SUB-GOALS: PROVE HASDUST, AND PROVE TARGETVISIBLE. EACH IS CHECKED AGAINST THE SENSORS AND THE KNOWLEDGE BASE.
- IF BOTH HASDUST AND TARGETVISIBLE ARE TRUE, THEN CANATTACK IS PROVEN, AND BECAUSE CANATTACK IS PROVEN, WINTEAMFIGHT IS PROVEN AND THE GOAL SUCCEEDS.
- IF EITHER ONE IS FALSE, RULE 1 FAILS, AND SINCE NO OTHER RULE CONCLUDES CANATTACK THE WHOLE PROOF FAILS AND WINTEAMFIGHT CANNOT BE ESTABLISHED. USEFULLY, THE AGENT NOW KNOWS EXACTLY WHICH FACT WAS MISSING, FOR EXAMPLE THAT IT HAS NO DUST, AND CAN TRIGGER A SHOPPING ACTION TO SATISFY IT.
- THE OVERALL MECHANISM IS TO START FROM THE GOAL, FIND RULES WHOSE CONCLUSIONS, WHICH ARE THE RIGHT-HAND SIDES, MATCH IT, AND RECURSIVELY CONVERT EACH OF THAT RULE'S PREMISES INTO A NEW SUB-GOAL, UNTIL EVERY BRANCH TERMINATES EITHER IN A KNOWN FACT, WHICH IS SUCCESS, OR IN AN EXHAUSTED RULE BASE, WHICH IS FAILURE. THIS IS EFFECTIVELY A DEPTH-FIRST RECURSIVE SEARCH OVER THE RULE BASE.
- THE DIRECTION OF TRAVEL IS FROM THE GOAL DOWNWARDS TO THE FACTS REQUIRED TO SUPPORT IT, WHICH IS WHY IT IS CALLED TOP-DOWN OR GOAL-DRIVEN REASONING.

- ITS CHARACTER IS FOCUSED, BECAUSE IT ONLY EVER TOUCHES RULES THAT ARE RELEVANT TO THE SPECIFIC QUESTION BEING ASKED.
- ITS MAIN COST IS THAT IT MUST BE RE-RUN FROM SCRATCH FOR EVERY NEW QUERY, AND IT MAY RE-DERIVE THE SAME SUB-GOALS REPEATEDLY ACROSS DIFFERENT QUERIES.
- IT IS TYPICALLY USED FOR DECISION MAKING AND DIAGNOSIS, FOR EXAMPLE ANSWERING "SHOULD I ENGAGE IN THIS FIGHT?", AND IT IS THE STRATEGY USED BY PROLOG WHEN RESOLVING A QUERY.

COMPARISON OF THE TWO DIRECTIONS:

- FORWARD CHAINING IS TRIGGERED WHEN NEW DATA ARRIVES, WHEREAS BACKWARD CHAINING IS TRIGGERED WHEN A NEW GOAL IS SET.
- FORWARD CHAINING STARTS FROM THE KNOWN FACTS, WHEREAS BACKWARD CHAINING STARTS FROM THE QUERY OR GOAL.
- FORWARD CHAINING MOVES FROM PREMISES TO CONCLUSIONS, WHEREAS BACKWARD CHAINING MOVES FROM A CONCLUSION BACK TO THE PREMISES THAT WOULD SUPPORT IT.
- FORWARD CHAINING DERIVES EVERY CONSEQUENCE IT CAN, WHEREAS BACKWARD CHAINING DERIVES ONLY WHAT IS NEEDED TO SETTLE THE QUESTION.
- THE WEAKNESS OF FORWARD CHAINING IS THAT IT PRODUCES IRRELEVANT FACTS, WHILE THE WEAKNESS OF BACKWARD CHAINING IS THAT IT MAY REPEAT THE SAME SUB-GOAL WORK ACROSS SEPARATE QUERIES.
- IN THIS TUTORIAL, FORWARD CHAINING TRAVELLED FROM HASDUST AND TARGETVISIBLE, TO CANATTACK, TO WINTEAMFIGHT, WHILE BACKWARD CHAINING TRAVELLED FROM WINTEAMFIGHT, TO CANATTACK, TO HASDUST AND TARGETVISIBLE. BOTH TRACES USED THE SAME TWO RULES, AND ONLY THE DIRECTION OF TRAVEL DIFFERED.

MEMORY HOOKS

- REACHABLE IS NOT THE SAME AS FEASIBLE. A* SAYS THERE IS A ROAD, THE KNOWLEDGE BASE SAYS THE ROAD IS MINED. SEARCH PROPOSES, LOGIC DISPOSES.
- CNF CONVERSION IS TWO MOVES: KILL THE ARROW USING $A \Rightarrow B$ BECOMES NOT A OR B, THEN APPLY DE MORGAN TO THE BRACKET. SO $(S \text{ AND } M) \Rightarrow G$ BECOMES NOT S OR NOT M OR G.
- RESOLUTION NEVER PROVES DIRECTLY, IT REFUTES. ADD THE NEGATED GOAL, RESOLVE CLAUSES, AND REACHING THE EMPTY CLAUSE MEANS THE GOAL WAS TRUE ALL ALONG.
- UNIFICATION MEANS ONE RULE SERVING MANY ENEMIES. THE SUBSTITUTION X TO SPECTRE IS THE GLUE BETWEEN A GENERAL LAW AND A SPECIFIC PERCEPT.
- FORWARD CHAINING ASKS "WHAT DOES THIS DATA MEAN?", BACKWARD CHAINING ASKS "WHAT WOULD IT TAKE TO MAKE THIS TRUE?". SAME RULES, OPPOSITE DIRECTIONS.